

Data Design Patterns

Mahmoud Parsian

Ph.D. in Computer Science
Assistant Teaching Professor @Santa Clara University
email: mahmoud.parsian@yahoo.com
last updated: 9/11/2026

Contents

0. Introduction	1
1. Summarization Patterns	1
2. In-Mapper-Combiner Design Pattern	6
3. Filtering Patterns	9
4. Organization Patterns	11
5. Join Patterns	16
6. Meta Patterns	16
7. Input/Output Patterns	16
8. References	17

0. Introduction

The goal of this paper/chapter is to present Data Design Patterns in an informal way.

The emphasis has been on pragmatism and practicality.

Data Design Patterns formats:

- [PDF version](#)
- [Markdown version](#)

Source code for Data Design Patterns is provided on [GitHub](#).

Data Design Patterns can be categorized as:

- 1. Summarization Patterns
- 2. In-Mapper-Combiner Pattern
- 3. Filtering Patterns
- 4. Organization Patterns
- 5. Join Patterns
- 6. Meta Patterns
- 7. Input/Output Patterns
- 8. References

1. Summarization Patterns

Numerical summarizations are a big part of Summarization Patterns. Numerical summarizations are patterns that involve calculating aggregate statistical values (minimum, maximum, average, median, standard deviation, ...) over data. If the data has keys (such as a department identifier, gene identifier, or patient identifier), then the goal is to group records by a key field and then calculate aggregates per group such

as minimum, maximum, average, median, or standard deviation. If the data does not have keys, then you compute the summarization over the entire data set without grouping.

The main purpose of summarization patterns is to summarize lots of data into meaningful data structures such as tuples, lists, and dictionaries.

Some Numerical Summarization patterns can be expressed directly in SQL. For example, let `gene_samples` be a table of (`gene_id`, `patient_id`, `biomarker_value`). Further, assume that we have about 100,000 unique `gene_id`(s), and that a patient (represented as `patient_id`) may have many gene records, each with an associated `biomarker_value`.

This Numerical Summarization pattern corresponds to using `GROUP BY` in SQL, for example:

```
SELECT MIN(biomarker_value), MAX(biomarker_value), COUNT(*)
FROM gene_samples
GROUP BY gene_id;
```

Therefore, in this example, we find a triplet (`min`, `max`, `count`) per `gene_id`.

In Spark, this summarization pattern can be implemented by using RDDs and DataFrames. In Spark, (`key`, `value`) pair RDDs are commonly used to `group by` a key (in our example `gene_id`) in order to calculate aggregates per group.

Let's assume that the input files are CSV file(s) and further assume that input records have the following format:

```
<gene_id><,><patient_id><,><biomarker_value>
```

RDD Example using `groupByKey()`

We load data from CSV file(s) and then create an `RDD[(key, value)]`, where key is `gene_id` and value is a `biomarker_value`. To solve it with `groupByKey()`, we first need to map it to a (`min`, `max`, `count`) triplet per value:

```
# rdd : RDD[(gene_id, biomarker_value)]
mapped = rdd.mapValues(lambda v: (v, v, 1))
```

Then we can apply the `groupByKey()` transformation:

```
# grouped : RDD[(gene_id, Iterable<biomarker_value>)]
grouped = mapped.groupByKey()

# calculate min, max, count for values
triplets = grouped.mapValues(
    lambda values: (min(values), max(values), len(values))
)
```

`groupByKey()` might cause OOM errors if you have too many values per key (`gene_id`), since `groupByKey()` does not use any combiners at all. Overall, `reduceByKey()` is a better scale-out solution than `groupByKey()`.

RDD Example using `reduceByKey()`

We load data from CSV file(s) and then create an `RDD[(key, value)]`, where key is `gene_id` and value is a `biomarker_value`. To solve it with `reduceByKey()`, we first need to map it to a (`min`, `max`, `count`) triplet:

```
# rdd : RDD[(gene_id, biomarker_value)]
mapped = rdd.mapValues(lambda v: (v, v, 1))
```

Then we can apply the `reduceByKey()` transformation:

```
# x = (min1, max1, count1)
# y = (min2, max2, count2)
reduced = mapped.reduceByKey(
    lambda x, y: (min(x[0], y[0]), max(x[1], y[1]), x[2] + y[2])
)
```

Spark's `reduceByKey()` merges the values for each key using an associative and commutative reduce function.

RDD Example using `combineByKey()`

We load data from CSV file(s) and then create an `RDD[(key, value)]`, where key is `gene_id` and value is a `biomarker_value`.

`RDD.combineByKey(createCombiner, mergeValue, mergeCombiners)` is a generic function to combine the elements for each key using a custom set of aggregation functions. It turns an `RDD[(K, V)]` into a result of type `RDD[(K, C)]`, for a “combined type” `C`. Note that, depending on your data requirements, the combined data type can be a simple data type (such as integer, string, ...), a collection (such as set, list, tuple, array, or dictionary), or a custom data type.

Users provide three functions:

1. `createCombiner`:
turns a `V` into a `C` (e.g., creates a one-element list)
2. `mergeValue`:
merges a `V` into a `C` (e.g., adds it to the end of a list)
3. `mergeCombiners`:
combines two `C`'s into a single one (e.g., merges the lists)

Here is the solution using `combineByKey()`:

```
# rdd : RDD[(gene_id, biomarker_value)]
combined = rdd.combineByKey(
    lambda v: (v, v, 1),
    lambda C, v: (min(C[0], v), max(C[1], v), C[2] + 1),
    lambda C, D: (min(C[0], D[0]), max(C[1], D[1]), C[2] + D[2])
)
```

DataFrame Example

After reading the input, we can create a `DataFrame` as: `DataFrame[(gene_id, patient_id, biomarker_value)]`.

```
# df : DataFrame[(gene_id, patient_id, biomarker_value)]
import pyspark.sql.functions as F

result = df.groupBy("gene_id") \
    .agg(F.min("biomarker_value").alias("min"),
         F.max("biomarker_value").alias("max"),
         F.count("biomarker_value").alias("count"))
```

An alternative solution is to use pure SQL: register your `DataFrame` as a temporary view, and then run a SQL query:

```
# register DataFrame as a temporary view named gene_samples
df.createOrReplaceTempView("gene_samples")

# find the result by SQL query:
query = """SELECT MIN(biomarker_value),
```

```

        MAX(biomarker_value),
        COUNT(*)
    FROM gene_samples
    GROUP BY gene_id"""

```

```
result = spark.sql(query)
```

Note that your SQL statement will be executed as a series of mappers and reducers behind the scenes by the Spark engine.

Data Without Keys - using DataFrames

You might have some numerical data without keys, and be interested in computing statistics such as (`min`, `max`, `count`) over the entire data set. In these situations, you have more than one option: you can use the `mapPartitions()` transformation or the `reduce()` action (depending on the format and nature of the input data), or simply use Spark's built-in aggregate functions.

Here's how to get the mean and standard deviation using Spark's built-in functions:

```

# import required functions
from pyspark.sql.functions import col
from pyspark.sql.functions import mean as _mean
from pyspark.sql.functions import stddev as _stddev

# apply desired functions
collected_stats = df.select(
    _mean(col('numeric_column_name')).alias('mean'),
    _stddev(col('numeric_column_name')).alias('stddev')
).collect()

# extract the final results:
final_mean = collected_stats[0]['mean']
final_stddev = collected_stats[0]['stddev']

```

Data Without Keys - using RDDs

If you have to filter your numeric data and perform other calculations before computing the mean and std-dev, then you may use the `RDD.mapPartitions()` transformation. The `RDD.mapPartitions(f)` transformation returns a new RDD by applying a function `f()` to each partition of this RDD. Finally, you may `reduce()` the result of the `RDD.mapPartitions(f)` transformation.

To understand the `RDD.mapPartitions(f)` transformation, let's assume that the input is a set of files, where each record has a set of numbers separated by commas (note that each record may have any number of numbers: for example, one record may have 5 numbers and another record might have 34 numbers, etc.):

```
<number><,><number><,>...<,><number>
```

Suppose the goal is to find (`min`, `max`, `count`, `num_of_negatives`, `num_of_positives`) for the entire data set. One easy solution is to use `RDD.mapPartitions(f)`, where `f()` is a function that returns (`min`, `max`, `count`, `num_of_negatives`, `num_of_positives`) per partition. Once `mapPartitions()` is done, we can apply a final reducer to find the overall (`min`, `max`, `count`, `num_of_negatives`, `num_of_positives`) across all partitions.

Let `rdd` denote `RDD[String]`, which represents all input records.

First, we define a helper function `count_neg_pos()`, which accepts a list of numbers and returns the count of negative and positive numbers:

```
def count_neg_pos(numbers):
    neg_count, pos_count = 0, 0
    # iterate numbers
    for num in numbers:
        if num > 0: pos_count += 1
        if num < 0: neg_count += 1
    #end-for
    return (neg_count, pos_count)
#end-def
```

Next, we define our custom function `compute_stats()`, which accepts a partition (an iterator of records) and returns (min, max, count, num_of_negatives, num_of_positives) for that partition:

```
def compute_stats(partition):
    first_time = True
    for e in partition:
        numbers = [int(x) for x in e.split(',') if x]
        neg_pos = count_neg_pos(numbers)
        #
        if first_time:
            _min = min(numbers)
            _max = max(numbers)
            _count = len(numbers)
            _neg = neg_pos[0]
            _pos = neg_pos[1]
            first_time = False
        else:
            # it is not the first time:
            _min = min(_min, min(numbers))
            _max = max(_max, max(numbers))
            _count += len(numbers)
            _neg += neg_pos[0]
            _pos += neg_pos[1]
        #end-if
    #end-for
    return [_min, _max, _count, _neg, _pos]
#end-def
```

After defining the `compute_stats(partition)` function, we can now apply the `mapPartitions()` transformation:

```
mapped = rdd.mapPartitions(compute_stats)
```

Now `mapped` is an `RDD[(int, int, int, int, int)]`.

Next, we can apply a final reducer to find the overall (min, max, count, num_of_negatives, num_of_positives) across all partitions:

```
tuple5 = mapped.reduce(lambda x, y: (min(x[0], y[0]),
                                     max(x[1], y[1]),
                                     x[2] + y[2],
                                     x[3] + y[3],
                                     x[4] + y[4])
                       )
```

Spark is so flexible and powerful that we might find multiple solutions for a given problem. But which is the optimal solution? This can be settled by testing your solutions against real data, similar to what you might use in production. Test, test, and test.

2. In-Mapper-Combiner Design Pattern

In this section, I will discuss the In-Mapper-Combiner design pattern and show some examples of using it.

In a typical MapReduce paradigm, mappers emit (**key**, **value**) pairs and once all mappers are done, the “sort and shuffle” phase prepares input (from the mappers’ output) for reducers in the form of (**key**, **Iterable<value>**). Finally, reducers consume these pairs and create the final (**key**, **aggregated-value**). For example, if mappers have emitted the following (**key**, **value**) pairs:

```
(k1, v1), (k1, v2), (k1, v3), (k1, v4),  
(k2, v5), (k2, v6)
```

Then “sort and shuffle” will prepare the following (**key**, **value**) pairs to be consumed by reducers:

```
(k1, [v1, v2, v3, v4])  
(k2, [v5, v6])
```

For some data applications, it is very possible to emit too many (**key**, **value**) pairs, which may create huge network traffic in the cluster. If a mapper creates the same key multiple times (with different values) for the input partition, then for some aggregation algorithms it is possible to aggregate/combine these (**key**, **value**) pairs and emit fewer of them. The simplest case is counting DNA bases (count A’s, T’s, C’s, G’s). In a typical MapReduce paradigm, for a DNA string of “AAAATTTCCCGGAAATGG”, a mapper will create the following (**key**, **value**) pairs:

```
(A, 1), (A, 1), (A, 1), (A, 1), (T, 1), (T, 1), (T, 1),  
(C, 1), (C, 1), (C, 1), (G, 1), (G, 1), (A, 1), (A, 1),  
(A, 1), (T, 1), (G, 1), (G, 1)
```

For this specific algorithm (DNA base count), it is possible to combine values for the same key:

```
(A, 7), (T, 4), (C, 3), (G, 4)
```

Combining/merging/reducing 18 (**key**, **value**) pairs into 4 combined (**key**, **value**) pairs is called “In-Mapper-Combining”: we combine values during mapper processing. The advantage is that we emit/create far fewer (**key**, **value**) pairs, which eases the cluster’s network traffic. The In-Mapper-Combiner emits (A, 7) instead of seven pairs of (A, 1), and so on. The In-Mapper-Combiner design pattern was introduced to address an issue with the MapReduce programming paradigm: limiting the number of (**key**, **value**) pairs generated by mappers.

When do you need the In-Mapper-Combiner design pattern? When your mapper generates too many (**key**, **value**) pairs and you have a chance to combine these pairs into a smaller number of (**key**, **value**) pairs, then you may use the In-Mapper-Combiner design pattern.

Informally, say that your mapper has created 3 keys with multiple (**key**, **value**) pairs as:

```
key k1: (k1, u1), (k1, u2), (k1, u3), ...  
key k2: (k2, v1), (k2, v2), (k2, v3), ...  
key k3: (k3, t1), (k3, t2), (k3, t3), ...
```

Then the In-Mapper-Combiner design pattern should combine these into the following (**key**, **value**) pairs:

```
(k1, combiner_function([u1, u2, u3, ...]))  
(k2, combiner_function([v1, v2, v3, ...]))  
(k3, combiner_function([t1, t2, t3, ...]))
```

Where `combiner_function([a1, a2, a3, ...])` is a custom function that combines/reduces `[a1, a2, a3, ...]` into a single value.

Applying the In-Mapper-Combiner design pattern may result in a more efficient algorithm implementation from a performance point of view (for example, reducing time complexity). The `combiner_function()` must guarantee that it is a semantic-preserving function, meaning that the semantics/correctness of the algorithm (with and without In-Mapper-Combiner) for mappers must not change at all. The In-Mapper-Combiner

design pattern can substantially reduce both the number and the size of the (key, value) pairs that need to be shuffled from the mappers to the reducers.

Example: DNA Base Count Problem

What is DNA Base Counting? The four bases in a DNA molecule are adenine (A), cytosine (C), guanine (G), and thymine (T). So a DNA string is comprised of the 4 base letters {A, T, C, G}. DNA Base Count finds the frequency of these base letters for a given set of DNA strings.

Input Format: FASTA

There are multiple text formats for representing DNA. FASTA is a text-based format for representing DNA data. The FASTA file format is a widely used format for specifying biosequence information. A sequence in FASTA format begins with a single description line, followed by one or more lines of sequence data.

Therefore, a FASTA file has two kinds of records:

- records that begin with >, which is a description line (should be ignored for DNA base count)
- records that do not begin with >, which is a DNA string

We will ignore the description records and focus only on DNA strings.

[Example](#) of two FASTA-formatted sequences in a file:

```
>NM_012514 Rattus norvegicus breast cancer 1 (Brca1), mRNA
CGCTGGTGCAACTCGAAGACCTATCTCCTTCCCGGGGGGGCTTCTCCGGCATTAGGCCT
CGGCGTTTGGAAGTACGGAGGTTTTTCTCGGAAGAAAGTTCACTGGAAGTGAAGAAATG
GATTTATCTGCTGTTTCAATTCGAATCAAGAAGTACAAAATGTCCTTCATGCTATGCAGAAAATC
TTGGAGTGTCCAATCTGTTTGAAGTATCAAGAAGTCAAGAAACCGGTTTCCACACAGTGCAGACCAC
ATATTTTGCAAAATTTGTATGCTGAAATCCTTAACCAGAAGAAAGGACCTTCCAGTGT
CCTTTGTGTAAGAATGAGATAACCAAAAGGAGCCTACAAGGAAGTGCAAGG
>NM_012515
TGTGGATCTTTCCAGAACAGCAGTTGCAATCACTATGTCTCAATCCTGGGTACCCGCCGT
GGGCCTCACTCTGGTGCCAGCCTGGGGGGCTTCATGGGAGCCTACTTTGTGCGTGGTGA
GGGCCTCCGCTGGTATGCTAGCTTGCAAGAACCTCCTGGCATCCGCCTCGCTGGACACT
CGTCCCCTCTGGGGCACACTGTATTCGGCCATGGGGTATGGCTCCTACATAATCTGGAA
AGAGCTGGGAGGTTTTCACAGAGGAGGCTATGGTTCCCTTGGGTCTCTACACTGGTCAGCT
```

Note that for all three solutions, we will drop description records (those beginning with the > symbol) by using the `RDD.filter()` transformation.

Solution 1: Classic MapReduce Algorithm

In the canonical example of DNA base counting, a (key, value) pair is emitted for every DNA base letter found, where the key is a DNA base letter in {A, T, C, G} and the value is 1 (a frequency of one). This solution creates too many (key, value) pairs. After mapping is done, we have several options for reducing these (key, value) pairs:

- Use `groupByKey()`
- Use `reduceByKey()`
- Use `combineByKey()`

Pros of Solution 1

- Simple solution, which works
- Mappers are fast; no need for combining counters

Cons of Solution 1

- Too many (key, value) pairs are created
- Might cause cluster network traffic

Solution 2: In-Mapper-Combiner Algorithm

In this solution, we will use the In-Mapper-Combiner design pattern, and per DNA string we will emit at most four (key, value) pairs:

```
(A, n1)
(T, n2)
(C, n3)
(G, n4)
```

where

```
n1: is the total frequency of A's per mapper input
n2: is the total frequency of T's per mapper input
n3: is the total frequency of C's per mapper input
n4: is the total frequency of G's per mapper input
```

To implement the In-Mapper-Combiner design pattern, we will use Python's `collections.Counter()` to keep track of DNA base letter frequencies. The other option is to use four variables (initialized to zero) and increment them as we iterate/scan the DNA string. Since the number of keys is very small (4 of them), it is easier to use 4 variables for counting; otherwise (when you have many keys) you should use a `collections.Counter()` to keep track of the frequency of each key.

Similar to Solution 1, we may apply any of the following reducers to find the final DNA base count:

- Use `groupByKey()`
- Use `reduceByKey()`
- Use `combineByKey()`

Pros of Solution 2

- Far fewer (key, value) pairs are created compared to Solution 1
- The In-Mapper-Combiner design pattern is applied
- Will not cause cluster network traffic, since there are not too many (key, value) pairs

Cons of Solution 2

- A dictionary is created per mapper; if we have too many mappers running concurrently, there might be an OOM error

Solution 3: Mapping Partitions Algorithm

This solution uses the `RDD.mapPartitions()` transformation to solve the DNA base count problem. In this solution we will emit four (key, value) pairs:

```
(A, p1)
(T, p2)
(C, p3)
(G, p4)
```

where

```
p1: is the total frequency of A's per single partition
p2: is the total frequency of T's per single partition
p3: is the total frequency of C's per single partition
```

p4: is the total frequency of G's per single partition

Note that a single partition may have thousands or millions of FASTA records. For this solution, we will create a single `collections.Counter()` per partition (rather than per RDD element).

Pros of Solution 3

- Far fewer (key, value) pairs are created compared to Solutions 1 and 2
- The Map Partitions design pattern is applied
- Will not cause cluster network traffic, since there are not too many (key, value) pairs
- A single dictionary is created per partition. Since the number of partitions can be in the hundreds or thousands, this is not a problem at all
- This is the most scaled-out solution: basically, we summarize DNA base counting per partition: from each partition, we emit at most four (key, value) pairs

Cons of Solution 3

- None

Summary and conclusion

The In-Mapper-Combiner design pattern is one method for summarizing the output of mappers, and hence for possibly improving the speed of your MapReduce job by reducing the number of intermediary (key, value) pairs emitted from mappers to reducers. As noted, there are several ways to implement the In-Mapper-Combiner design pattern, depending on your mappers' input and expected output. One immediate benefit of the In-Mapper-Combiner design pattern is that it drastically reduces the number of (key, value) pairs emitted from mappers to reducers.

Download FASTA Files

1. [GeoSymbio Downloads](#)
2. [NCBI Downloads](#)

3. Filtering Patterns

Filter patterns are a set of design patterns that enable us to filter a set of records (or elements) using different criteria, and to chain them in a decoupled way through logical operations. One simple example is to filter out records if the salary of that record is less than 20,000. Another example is to filter out records that do not contain a valid URL. This type of design pattern falls under the structural pattern category, since it combines multiple criteria to obtain a single criterion.

For example, Python offers filtering as:

```
filter(function, sequence)
```

where

- **function:** a function that tests whether each element of a sequence is true or not.
- **sequence:** the sequence to be filtered; it can be a set, list, tuple, or any iterable container.

Returns: an iterator that has already been filtered.

A simple example is given below:

```
# function that filters DNA letters
def is_dna(variable):
    dna_letters = ['A', 'T', 'C', 'G']
    if variable in dna_letters:
        return True
```

```

        else:
            return False
#end-def

# sequence
sequence = ['A', 'B', 'T', 'T', 'C', 'G', 'M', 'R', 'A']

# using filter function
# filtered = ['A', 'T', 'T', 'C', 'G', 'A']
filtered = filter(is_dna, sequence)

```

PySpark offers filtering at large scale for both RDDs and DataFrames.

Filter using RDD

Let `rdd` be an `RDD[(String, Integer)]`. Assume the goal is to keep `(key, value)` pairs if and only if the value is greater than 0. This is pretty straightforward to accomplish in PySpark by using the `RDD.filter()` transformation:

```

# rdd: RDD[(String, Integer)]
# filtered: RDD[(key, value)], where value > 0
# e = (key, value)
filtered = rdd.filter(lambda e: e[1] > 0)

```

The filter can also be implemented with a named boolean predicate function:

```

def greater_than_zero(e):
    # e = (key, value)
    if e[1] > 0:
        return True
    else:
        return False
#end-def

# filtered: RDD[(key, value)], where value > 0
filtered = rdd.filter(greater_than_zero)

```

Filter using DataFrame

Filtering records using a DataFrame can be accomplished with `DataFrame.filter()`, or you may use `DataFrame.where()`.

Consider `df` as a `DataFrame[(emp_id, city, state)]`.

Then you may use the following filtering patterns:

```

# SparkSession available as 'spark'.
>>> tuples3 = [('e100', 'Cupertino', 'CA'), ('e200', 'Sunnyvale', 'CA'),
               ('e300', 'Troy', 'MI'), ('e400', 'Detroit', 'MI')]
>>> df = spark.createDataFrame(tuples3, ['emp_id', 'city', 'state'])
>>> df.show()
+-----+-----+-----+
|emp_id|    city|state|
+-----+-----+-----+
|  e100|Cupertino|   CA|
|  e200|Sunnyvale|   CA|
|  e300|    Troy|   MI|
|  e400|  Detroit|   MI|

```

```

+-----+-----+-----+

>>> df.filter(df.state != "CA").show(truncate=False)
+-----+-----+-----+
|emp_id|city  |state|
+-----+-----+-----+
|e300  |Troy   |MI   |
|e400  |Detroit|MI   |
+-----+-----+-----+

>>> df.filter(df.state == "CA").show(truncate=False)
+-----+-----+-----+
|emp_id|city      |state|
+-----+-----+-----+
|e100  |Cupertino|CA   |
|e200  |Sunnyvale|CA   |
+-----+-----+-----+

>>> from pyspark.sql.functions import col
>>> df.filter(col("state") == "MA").show(truncate=False)
+-----+-----+-----+
|emp_id|city|state|
+-----+-----+-----+

>>> df.filter(col("state") == "MI").show(truncate=False)
+-----+-----+-----+
|emp_id|city  |state|
+-----+-----+-----+
|e300  |Troy   |MI   |
|e400  |Detroit|MI   |
+-----+-----+-----+

```

You may also use the `DataFrame.where()` function to filter rows:

```

>>> df.where(df.state == 'CA').show()
+-----+-----+-----+
|emp_id|city|state|
+-----+-----+-----+
| e100|Cupertino| CA|
| e200|Sunnyvale| CA|
+-----+-----+-----+

```

For more examples, you may read [PySpark Where Filter Function | Multiple Conditions](#).

4. Organization Patterns

Organization Patterns deal with reorganizing data for use by other rendering applications. For example, you might have structured data in different formats and from different data sources, and you might join and merge that data into XML or JSON formats. Another example is partitioning data (the so-called binning pattern) based on some categorization (such as continent, country, ...).

4.1 The Structured-to-Hierarchical Pattern

The goal of this pattern is to convert structured data (in different formats and from different data sources) into a hierarchical (XML or JSON) structure. You need to bring all the data into a single location so that

you can convert it into a hierarchical structure. In a nutshell, the Structured-to-Hierarchical pattern creates new hierarchical records (such as XML or JSON) from data that started out in a very different structure (flat/plain records). The main objective of the Structured- to-Hierarchical pattern is to transform row-based data into a hierarchical format (such as JSON or XML).

For example, consider blog data with comments from many users. A hierarchy will look something like:

```
Posts
  Post-1
    Comment-11
    Comment-12
    Comment-13
  Post-2
    Comment-21
    Comment-22
    Comment-23
    Comment-24
  ...
```

Assume that there are two types of structured data, which can be joined to create the hierarchical structure shown above.

Data Set 1:

```
<post_id><,><title><,><creator>
```

Example of Data Set 1 records:

```
p1,t1,creator1
p2,t2,creator2
p3,t3,creator3
...
```

Data Set 2:

```
<post_id><,><comment><,><commented_by>
```

Example of Data Set 2 records:

```
p1,comment-11,commentedby-11
p1,comment-12,commentedby-12
p1,comment-13,commentedby-13
p2,comment-21,commentedby-21
p2,comment-22,commentedby-22
p2,comment-23,commentedby-23
p2,comment-24,commentedby-24
...
```

Therefore, the goal is to join and merge these two data sets so that we can create an XML document for a single post, such as:

```
<post id="p1">
  <title>t1</title>
  <creator>creator1</creator>
  <comments>
    <comment>comment-11</comment>
    <comment>comment-12</comment>
    <comment>comment-13</comment>
  </comments>
</post>
```

```

<post id="p2">
  <title>t2</title>
  <creator>creator2</creator>
  <comments>
    <comment>comment-21</comment>
    <comment>comment-22</comment>
    <comment>comment-23</comment>
    <comment>comment-24</comment>
  </comments>
</post>
...

```

I will provide two solutions: RDD-based and DataFrame-based.

RDD Solution Step 1: This solution reads the data sets and creates two RDDs keyed by `post_id`:

```

posts: RDD[(post_id, (title, creator))]
comments: RDD[(post_id, (comment, commented_by))]

```

Step 2: These two RDDs are joined by the common key `post_id`:

```

# joined: RDD[(post_id, ((title, creator), (comment, commented_by)))]
joined = posts.join(comments)

```

Step 3: Apply a reducer: group by `post_id`:

```

# grouped = RDD[(post_id, Iterable<((title, creator), (comment, commented_by))>)]
grouped = joined.groupByKey()

```

Step 4: The final step is to iterate over the `grouped` elements and create the XML (or JSON):

```

xml_rdd = grouped.map(create_xml)

# where
#
# element: (post_id, Iterable<((title, creator), (comment, commented_by))>)
def create_xml(element):
    xml = ... # perform concatenation of required items and build the desired XML
    return xml
#end-def

```

A complete example implementation is given in: `structured_to_hierarchical_to_xml_rdd.py`

DataFrame Solution In the DataFrame solution, we read the data sets and create DataFrames:

```

posts = spark.createDataFrame(posts_data, ["post_id", "title", "creator"])
comments = spark.createDataFrame(comments_data, ["post_id", "comment", "commented_by"])

```

Next, these two DataFrames are joined on the common key `post_id`, and then we select the required columns:

```

joined_and_selected = posts.join(comments, posts.post_id == comments.post_id) \
    .select(posts.post_id, posts.title, posts.creator, comments.comment)

```

Next, we group the result by ("`post_id`", "`title`", "`creator`") and collect the `comment` column values into a list:

```

grouped = joined_and_selected.groupBy("post_id", "title", "creator") \
    .agg(F.collect_list("comment").alias("comments"))

```

To create the XML, we use a UDF:

```
create_xml_udf = F.udf(
    lambda post_id, title, creator, comments:
        create_xml(post_id, title, creator, comments),
    StringType()
)
```

Finally, we apply the UDF to the proper columns to create the XML:

```
df = grouped.withColumn(
    "xml",
    create_xml_udf(grouped.post_id, grouped.title, grouped.creator, grouped.comments)
) \
    .drop("title") \
    .drop("creator") \
    .drop("comments")
```

A complete example implementation is given in: `structured_to_hierarchical_to_xml_dataframe.py`

4.2 The Partitioning and Binning Pattern

Bucketing, binning, and categorization of data are used synonymously in technical papers and blogs. Data binning, also called discrete binning or bucketing, is a data pre-processing technique used to reduce the effects of minor observation errors. Binning is a way to group a number of more- or-less continuous values into a smaller number of “bins.” For example, if you have data about a group of graduated students, with a number of years of education, then you might categorize it as HSDG (12 years), AA (14 years), BS (16 years), MS (18 years), PhD (21 years), or MD (22+ years). This creates 6 bins: {HSDG, AA, BS, MS, PhD, MD}. Once the bins are created, you can use categorical values (HSDG, BS, ...) in your data queries.

The original data values that fall into a given small interval — a bin — are replaced by a value representative of that interval, often the central value. For example, if a car’s price is highly scattered, you may use bucketing instead of the actual car prices.

[Spark’s Bucketizer](#) transforms a column of continuous features into a column of feature buckets, where the buckets are specified by the user.

Consider this example: there is no linear relationship between latitude and housing values, but individual latitudes and housing values may still be related, even though the relationship is not linear. Therefore, you might bucketize the latitudes; for example, you may create buckets such as:

```
Bin-1: 32 < latitude <= 33
Bin-2: 33 < latitude <= 34
...
```

The binning technique can be applied to both categorical and numerical data. The following examples show both types of binning.

Numerical Binning Example:

value	Bin
0-10	Very Low
11-30	Low
31-70	Mid
71-90	High
91-100	Very High

Categorical Binning Example

value	Bin
India	Asia
China	Asia
Japan	Asia
Spain	Europe
Italy	Europe
Chile	South America
Brazil	South America

Binning is also used in genomics: we bucketize human genome chromosomes (1, 2, 3, ..., 22, X, Y, MT). For instance, chromosome 1 has 250 million positions, which we may bucketize into 101 buckets as:

```
for id in (1, 2, 3, ..., 22, 'X', 'Y', 'MT'):
    chr_position = ... # chromosome-<id> position
    # chr_position range is from 1 to 250,000,000
    bucket = chr_position % 101
    # where
    #      0 <= bucket <= 100
```

Bucketing is the most straightforward approach for converting continuous variables into categorical variables. To understand this, let's look at an example below. In PySpark, the task of bucketing can be easily accomplished using the `Bucketizer` class.

To use the `Bucketizer` class, we first need to define the bucket borders. Let's define a list of bucket borders, as in the example below. Next, we create an object of the `Bucketizer` class, and then apply its `transform()` method to our `DataFrame dataframe`.

First, let's create a sample `DataFrame` for demo purposes:

```
>>> data = [('A', -99.99), ('B', -0.5), ('C', -0.3),
...         ('D', 0.0), ('E', 0.7), ('F', 99.99)]
>>> column_names = ["id", "features"]
>>> dataframe = spark.createDataFrame(data, column_names)
>>> dataframe.show()
+----+-----+
| id | features |
+----+-----+
| A  | -99.99  |
| B  | -0.5    |
| C  | -0.3    |
| D  | 0.0     |
| E  | 0.7     |
| F  | 99.99   |
+----+-----+
```

Next, we apply the `Bucketizer` to create buckets:

```
>>> bucket_borders = [-float("inf"), -0.5, 0.0, 0.5, float("inf")]
>>> from pyspark.ml.feature import Bucketizer
>>> bucketer = Bucketizer().setSplits(bucket_borders) \
...     .setInputCol("features").setOutputCol("bucket")
>>> bucketer.transform(dataframe).show()
+----+-----+-----+
| id | features | bucket |
+----+-----+-----+
```

	A	-99.99	0.0
	B	-0.5	1.0
	C	-0.3	1.0
	D	0.0	2.0
	E	0.7	3.0
	F	99.99	3.0

5. Join Patterns

Join patterns combine records from two (or more) data sets that share a common key, such as `post_id` in Section 4.1 or `gene_id` in Section 1. In Spark, the most common physical join strategies are:

- **Shuffle hash join** — both sides are shuffled/partitioned by the join key, and a hash join is performed per partition. This is the default strategy for large-to-large joins.
- **Sort-merge join** — both sides are shuffled and sorted by the join key, then merged. Spark’s default for large-to-large equi-joins.
- **Broadcast hash join** — the smaller data set is broadcast to every executor, avoiding a shuffle of the larger data set entirely. Ideal when one side of the join is small enough to fit in memory (see `spark.sql.autoBroadcastJoinThreshold`).

Choosing the right join strategy (and avoiding data skew on the join key) is critical for performance at scale. Join Patterns are covered in depth in Chapter 11 of [Data Algorithms with Spark](#).

6. Meta Patterns

Metadata is data that describes and gives information about other data. Meta Patterns are about “patterns that deal with patterns.” For example, in the MapReduce paradigm, “job chaining” is a meta pattern that pieces together several patterns to solve complex data problems. Another meta pattern is “job merging,” an optimization for performing several data analytics tasks within the same MapReduce job — effectively executing multiple MapReduce jobs as a single job.

Spark is a superset of the MapReduce paradigm and deals with meta patterns in terms of estimators, transformers, and pipelines, which are discussed here:

- [ML Pipelines](#)
- [Want to Build Machine Learning Pipelines?](#)

7. Input/Output Patterns

Input/Output Patterns address how data enters and leaves a Spark job: which file formats to read and write, how to control the number and size of output files, and how to organize output so downstream jobs can consume it efficiently.

7.1 Input Patterns

Spark can read data from many structured and semi-structured formats — text, CSV, JSON, Parquet, ORC, Avro, and JDBC sources, among others — using `spark.read.<format>(...)` or `sc.textFile(...)` for plain text/RDDs. A few practical considerations:

- **Splittable vs. non-splittable input.** Plain text and formats compressed with a splittable codec (such as `bzip2`) can be divided into many partitions and processed in parallel. Some compression codecs (such as plain `gzip`) are *not* splittable, meaning a single compressed file is read entirely by a single task — this can create a performance bottleneck when a handful of large `.gz` files dominate the input.

- **Schema-on-read vs. explicit schema.** Formats such as JSON and CSV support automatic schema inference (`spark.read.option("inferSchema", "true")`), which is convenient but requires an extra pass over the data and can misinfer types. For production jobs, prefer supplying an explicit `StructType` schema — it is faster and more predictable.
- **Columnar formats for repeated analytics.** When the same data set will be scanned repeatedly, columnar formats such as Parquet or ORC are strongly preferred over row-based formats such as CSV or JSON: they support column pruning, predicate pushdown, and efficient compression, all of which can dramatically reduce I/O.
- **Small-files problem on read.** Reading a directory with a very large number of small files creates one task per file (or per small group of files), causing excessive scheduling overhead. Consider `spark.sql.files.maxPartitionBytes` and `spark.sql.files.openCostInBytes` to coalesce small files into fewer, larger read tasks.

7.2 Output Patterns

- **Partitioned output.** `DataFrameWriter.partitionBy(*cols)` writes data into a directory hierarchy keyed by one or more columns (e.g., `/output/year=2026/month=09/...`). This lets downstream readers prune entire directories via partition filters instead of scanning the full data set. Choose partition columns with a reasonable number of distinct values — over-partitioning (for example, on a high-cardinality column) produces the small-files problem described above.
- **Controlling the number of output files.** The number of output files per partition equals the number of RDD/DataFrame partitions at write time. Use `coalesce(n)` to reduce the number of partitions without a full shuffle, or `repartition(n)` (or `repartition(n, *cols)`) when you need an even, shuffled redistribution — for example, before writing with `partitionBy()`, to avoid one tiny file per task/partition combination.
- **Choosing an output format.** Prefer Parquet (or ORC) for data that will be consumed by other Spark/SQL jobs — they are compact, splittable, and schema-aware. Reserve CSV/JSON output for interoperability with external, non-Spark consumers.
- **Idempotent writes.** Use `mode("overwrite")` or `mode("append")` deliberately, and prefer writing to a new, temporary output path and then atomically renaming/swapping it into place, rather than overwriting a directory that a downstream job might be reading concurrently.

8. References

1. [Spark with Python \(PySpark\) Tutorial For Beginners](#)
2. [Data Algorithms with Spark](#), author: Mahmoud Parsian
3. [PySpark Algorithms](#), author: Mahmoud Parsian
4. [Apache PySpark Documentation](#)
5. [PySpark Tutorial](#), author: Mahmoud Parsian
6. J. Lin and C. Dyer. [Data-Intensive Text Processing with MapReduce](#)